

Bincom Cybersecurity Training

Attack and Defend Web Application: Deliverable Document

Name: CHUKWU PRAISEGOD

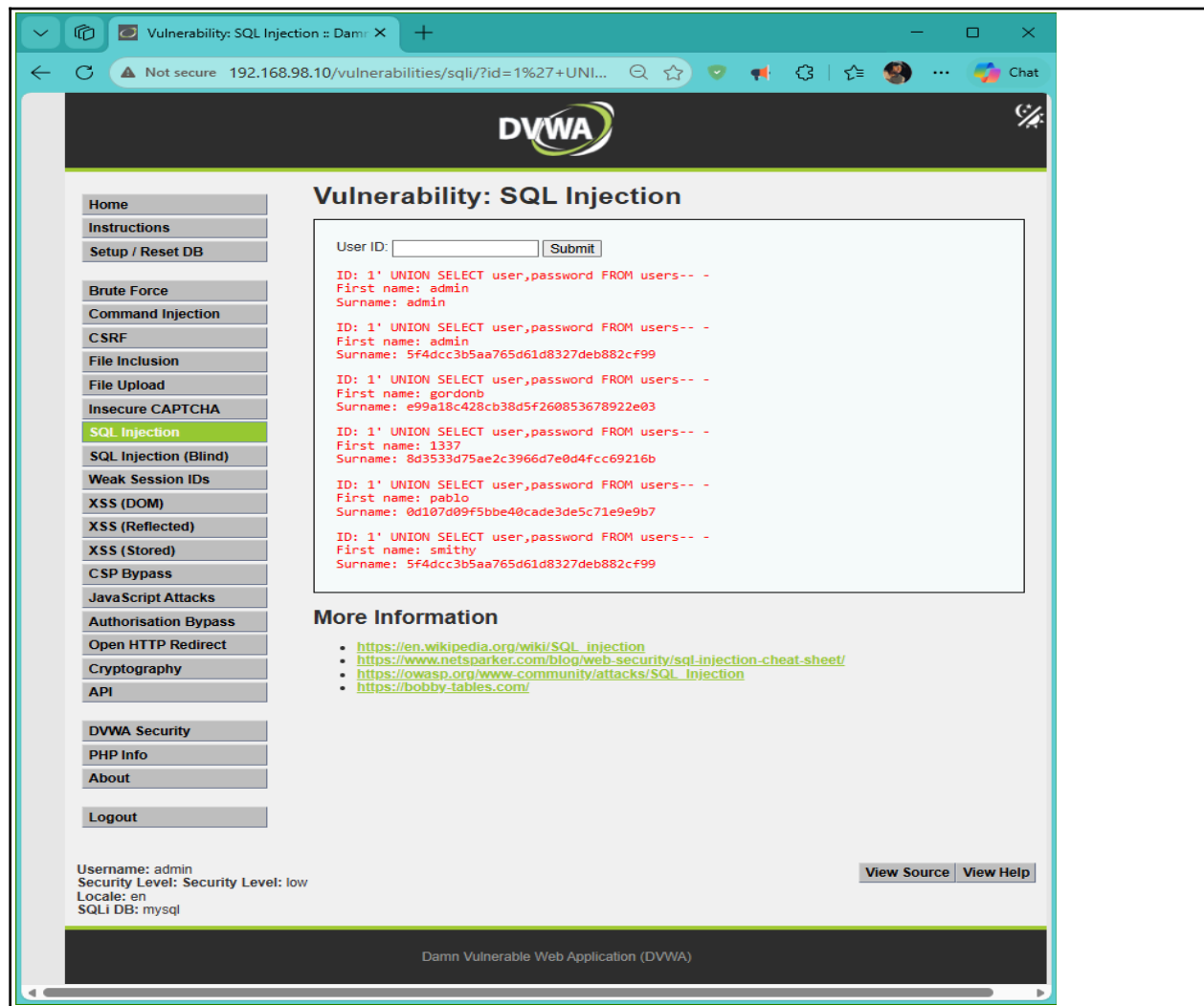
Date: 18/09/2026

Repository link: <https://github.com/chukwupg/dvwa-prestashop-security-audit>

1. Screenshots of Attack Success and Block, DVWA

The following screenshots demonstrate SQL Injection and Cross Site Scripting attacks executed against the unprotected DVWA instance, followed by the same attacks being blocked once the ModSecurity WAF was deployed and switched to enforcement mode.

1.1 SQL Injection, Before WAF



1.2 Cross Site Scripting, Before WAF

The screenshot shows a web browser window with two tabs, both titled 'Vulnerability: Stored Cross Site Scripting'. The address bar shows the URL '192.168.98.10/vulnerabilities/xss_s/'. A dark notification box at the top center displays the message '192.168.98.10 says Device is infected' with an 'OK' button. Below this, a form for signing the guestbook is visible, with a 'Message *' input field and 'Sign Guestbook' and 'Clear Guestbook' buttons. A red arrow points from the notification box to the 'Sign Guestbook' button. The left sidebar contains a list of vulnerability categories, with 'XSS (Stored)' highlighted in green. The main content area shows a form with 'Name: test' and 'Message: This is a test comment.' Below this, a red box highlights a form with 'Name: XSS Stored' and 'Message:'. The 'More Information' section lists several links related to XSS attacks.

192.168.98.10 says
Device is infected
OK

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout

Name: test
Message: This is a test comment.

Name: XSS Stored
Message:

Name: XSS Stored
Message:

More Information

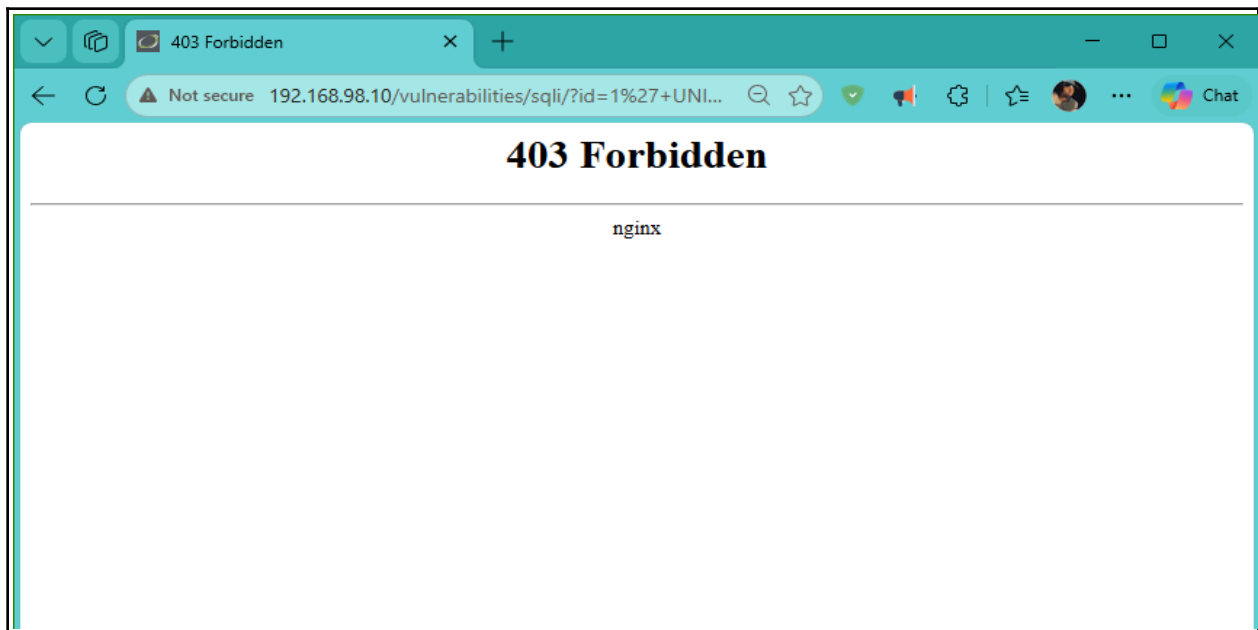
- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

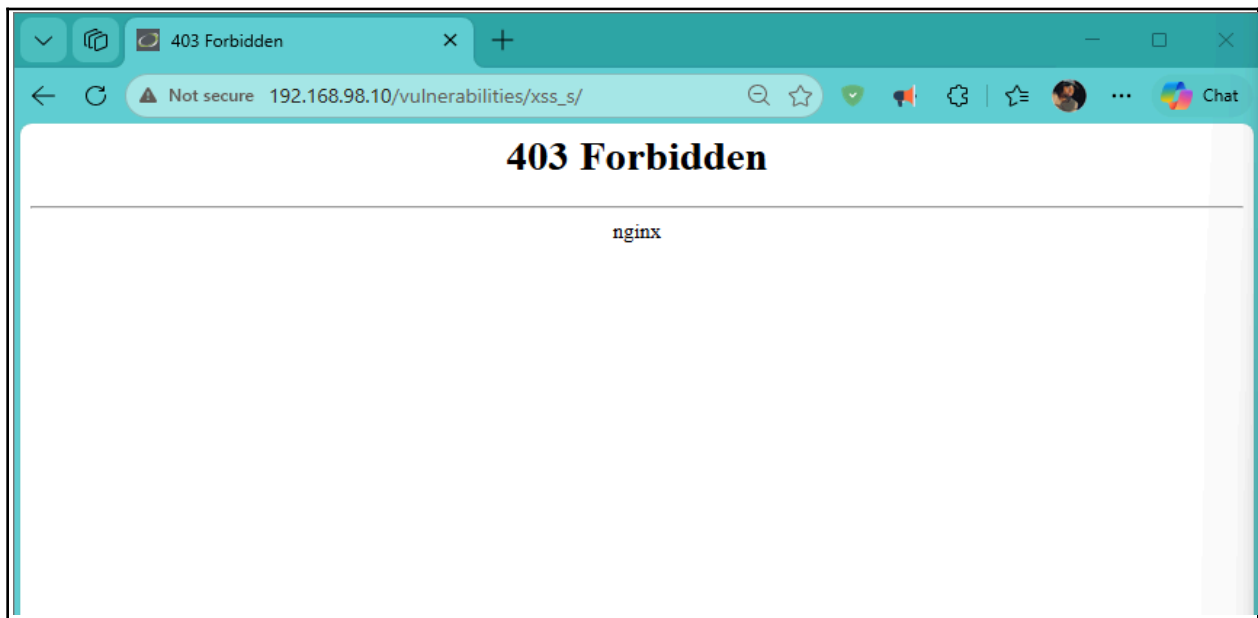
View Source View Help

Damn Vulnerable Web Application (DVWA)

1.3 SQL Injection, Blocked by WAF



1.4 Cross Site Scripting, Blocked by WAF



2. Configured WAF Rule File

I used ModSecurity with the OWASP Core Rule Set, deployed as an Nginx reverse proxy in front of DVWA, supplemented with a custom rule file written specifically to target the SQL Injection and Cross Site Scripting payloads used in this assignment.

Link to Custom rule file: <https://drive.google.com/file/d/18hxz-Zc31ix3QPIDxj-Wf9RptuxP9ASP/>

3. Short Report on Defense Validation

3.1 Objective

Demonstrate exploitation of SQL Injection and Cross Site Scripting vulnerabilities in DVWA, deploy a ModSecurity Web Application Firewall, and validate that the WAF blocks the previously successful attacks without altering legitimate application behavior.

3.2 Environment

- Target application: DVWA, ghcr.io/digininja/dvwa, security level Low
- Database: MariaDB 10, internal only, not exposed to the WAF or the network
- WAF: owasp/modsecurity-crs, Nginx plus ModSecurity version 3 plus OWASP Core Rule Set version 4, paranoia level 1
- Network: Docker bridge network on an Ubuntu server, isolated host only lab network with no direct internet exposure

3.3 Methodology

1. Executed SQL Injection and Cross Site Scripting against DVWA directly, with no WAF present, confirming both were exploitable.
2. Deployed ModSecurity and the OWASP Core Rule Set as a reverse proxy in front of DVWA, with custom rules targeting the exact payloads used.
3. Ran the WAF in DetectionOnly mode and repeated the same payloads, confirming the rules matched and were logged without altering the application response.
4. Switched the WAF to enforcement mode and repeated the same payloads again, confirming they were now blocked with a 403 Forbidden response.

3.4 Findings

SQL Injection: the id parameter in the DVWA SQL Injection module was exploited by enumerating the column count using ORDER BY, then extracting the full user table, including password hashes, using UNION SELECT.

Cross Site Scripting: both the reflected XSS module and the stored XSS guestbook module accepted a script tag payload that executed in the browser, with the stored variant persisting across page loads.

3.5 Mitigation Applied

- OWASP Core Rule Set version 4 as the baseline ruleset, covering the SQL Injection and Cross Site Scripting rule families
- Custom rules targeting ORDER BY enumeration, UNION SELECT extraction, tautology based bypass, script tag injection, inline event handler injection, and javascript URI injection
- Rules deployed ahead of the Core Rule Set using the before CRS hook point

3.6 Validation Results

Attack	Before WAF	WAF, DetectionOnly	WAF, Enforced
SQL Injection, ORDER BY enumeration	200 OK, exploited	200 OK, logged	403 Forbidden, blocked
SQL Injection, UNION SELECT extraction	200 OK, exploited	200 OK, logged	403 Forbidden, blocked
Cross Site Scripting, reflected	Alert executed	200 OK, logged	403 Forbidden, blocked
Cross Site Scripting, stored	Alert persisted	200 OK, logged	403 Forbidden, blocked

3.7 Conclusion

The deployed ModSecurity WAF, combining the OWASP Core Rule Set with targeted custom rules, successfully blocks the SQL Injection and Cross Site Scripting techniques demonstrated against the unprotected DVWA baseline. Detection was validated independently before enforcement was enabled, giving confidence that the blocking observed reflects accurate rule matching rather than incidental over blocking.

4. PrestaShop Security Checklist, Completed

Environment: this assessment was performed against the live PrestaShop lab instance originally deployed for Bincom Training Assignment 2. No new instance was stood up for this assignment. Real IP addresses, the database endpoint, and personal account details used during testing have been redacted.

1. Ensure application and modules are up to date, status: Fail

Current version is PrestaShop 8.2.1. The current patched release for this branch is 8.2.6, which fixes a critical stored Cross Site Scripting vulnerability in the back office Customer Service view, tracked as CVE 2026 44212. Nearly all installed modules are flagged as having an upgrade available in Module Manager, consistent with the outdated core.

Remediation: Update to 8.2.6 or later as soon as possible. Until patched, treat back office Customer Service access as elevated risk.

2. Remove or disable default and demo accounts and sample data, status: Partial

Accounts: pass. Only one employee account exists, active, using a real, non default email address. Catalog data: fail. All nineteen products in the catalog are unmodified default PrestaShop demo items, with reference codes such as demo_1 through demo_21.

Remediation: Bulk delete demo products via Catalog, Products, before this instance is used beyond lab or training purposes.

3. Verify strong admin password and MFA where possible, status: Partial

Password: pass. Eleven characters, mixed upper and lower case, numbers, and three symbols, which is reasonable strength though under the stricter twelve character bar some policies recommend. MFA: fail. No native two factor authentication section is available on employee account settings, and no two factor authentication module is installed.

Remediation: Consider extending the password to twelve or more characters. Evaluate a dedicated two factor authentication module, or enforce access control at the infrastructure layer through an IP allowlist on the admin path.

4. Review file and folder permissions and disable directory listing, status: Pass

No world writable directories or files were found under the web root. Key directories are owned by www data with permissions of 755 and 644, a standard, least privilege configuration. Directory listing is disabled, confirmed from outside the server, with the modules, img, and download paths all returning 302 redirects rather than raw file listings.

Remediation: None required.

5. Confirm secure session cookie settings, HttpOnly, Secure, SameSite, status: Partial

HttpOnly and SameSite are both correctly set on all session cookies, on the storefront and the admin panel. The Secure flag is absent on all cookies because the site is served over plain HTTP, and this flag cannot be meaningfully set without TLS.

Remediation: Once TLS is enabled, configure the application and server to enforce the Secure flag on all cookies. Note that because HttpOnly is correctly set, the impact of the stored Cross Site Scripting finding

in item one is partially reduced, since script based access to the session cookie is blocked even if a payload executes.

6. Disable unnecessary modules and plugins, status: Partial

No individually high risk modules were identified. The module set reflects an unmodified default install, consistent with the demo catalog finding in item two. Three payment modules, Bank Transfer, Cash on Delivery, and Payments by Check, are enabled simultaneously, none requiring live credentials. The Distribution API Client module is also enabled.

Remediation: Update core and modules together, as covered by the item one remediation. Disable the Distribution API Client module if native auto update is not in use. Reduce to a single real payment module before production use.

7. Validate secure configuration for payment and API endpoints, status: Pass

The Webservice and API feature is confirmed disabled in the admin panel, and the api endpoint returns 404 Not Found, meaning it is not routed at all rather than merely requiring authentication. No live payment gateway is in use, so there is no live credential exposure to assess.

Remediation: None required currently. Re verify this item if a real payment gateway is added in the future.

8. Ensure TLS is configured and enforced, status: Fail

HTTP on port 80 is reachable and returns 200 OK. HTTPS on port 443 times out, confirming nothing is listening on that port at all. No TLS certificate is present anywhere on the server for this application. The AWS security group already permits inbound traffic on port 443, so no infrastructure change is needed once a certificate is installed.

Remediation: Configure TLS using Let's Encrypt and Certbot if a domain name is available, or a self signed certificate for lab purposes. Enforce a redirect from HTTP to HTTPS and enable the PS_SSL_ENABLED setting once TLS is live.

9. Check for exposed admin paths and use IP restrictions or WAF rules, status: Partial

The default slash admin slash path correctly returns 404 Not Found, confirming the admin folder was renamed as part of prior hardening work. However, the renamed admin path is reachable from anywhere on the internet, the same as the public storefront, with no IP allowlist or additional access restriction specific to admin access, and no WAF is deployed in front of this instance at all.

Remediation: Restrict the admin path to trusted IP addresses using an Apache Location directive or a dedicated security group rule. Deploy a WAF in front of Apache, for example the same ModSecurity and OWASP Core Rule Set pattern used for DVWA in this assignment.

10. Backup and recovery verification, status: Partial

RDS automated backups are enabled, with a retention period of one day and a defined backup window. No EBS snapshot coverage exists for the EC2 application volume. The restore process has never been tested end to end.

Remediation: Increase RDS backup retention to between seven and thirty five days depending on requirements. Set up scheduled EBS snapshots for the application volume. Perform and document at least one full test restore.

Checklist Summary

Item	Status
1. Ensure application and modules are up to date	Fail
2. Remove or disable default and demo accounts and sample data	Partial
3. Verify strong admin password and MFA where possible	Partial
4. Review file and folder permissions and disable directory listing	Pass
5. Confirm secure session cookie settings, HttpOnly, Secure, SameSite	Partial
6. Disable unnecessary modules and plugins	Partial
7. Validate secure configuration for payment and API endpoints	Pass
8. Ensure TLS is configured and enforced	Fail
9. Check for exposed admin paths and use IP restrictions or WAF rules	Partial
10. Backup and recovery verification	Partial

Overall result: two items pass, six items partial, two items fail.

Planned Attack Simulation

Findings from items one and nine were used as the basis for the attack simulation described in section 5: an outdated version carrying a known critical stored Cross Site Scripting vulnerability, combined with an admin panel that has no IP restriction and no WAF protection.

5. PrestaShop Attack Simulation Report

5.1 Objective

Demonstrate practical, exploitable impact from the gaps identified in the completed security checklist, specifically targeting findings from checklist items one, three, eight, and nine.

5.2 Environment and Scope

Target: PrestaShop 8.2.1 on AWS, EC2 running Apache and PHP, RDS running MySQL, the lab instance originally deployed for Bincom Training Assignment 2. Testing was confined to this self owned lab instance, and no external systems were targeted.

5.3 Methodology

Checklist findings were reviewed for genuine exploitability, as opposed to hygiene only findings. Each selected vector was attempted against the live, unpatched instance, iterating on payloads or techniques where initial attempts were blocked or inconclusive. Evidence was captured for each attempt, including partial or inconclusive ones, since these still demonstrate real underlying weaknesses. Each finding was mapped back to its root cause and a specific, actionable mitigation.

5.4 Vector 1, Stored Cross Site Scripting via Contact Form Email Field, CVE 2026 44212

CVE 2026 44212, with a CVSS score of 9.3, critical, affects PrestaShop versions prior to 8.2.6 and 9.1.1. An unauthenticated attacker can submit the public Contact Us form with a maliciously crafted email address exploiting a permissive server side validator. The payload is stored and rendered without escaping in the back office Customer Service thread view, executing in an authenticated employee's browser session.

Root cause: PrestaShop's email validator correctly implements the RFC 5321 quoted string local part rule, a legitimate email syntax allowing the portion before the at symbol to be wrapped in double quotes and contain otherwise restricted characters. This is valid by specification. The actual flaw is that this data is later rendered in the admin interface without consistent HTML escaping.

Methodology: script tag payloads without quoting were rejected by both client side and server side validation. Wrapping the payload in an RFC 5321 quoted string, for example a quoted script tag followed by the at symbol and a test domain, was accepted and the message was sent successfully. The resulting thread was opened in the back office and examined using view page source.

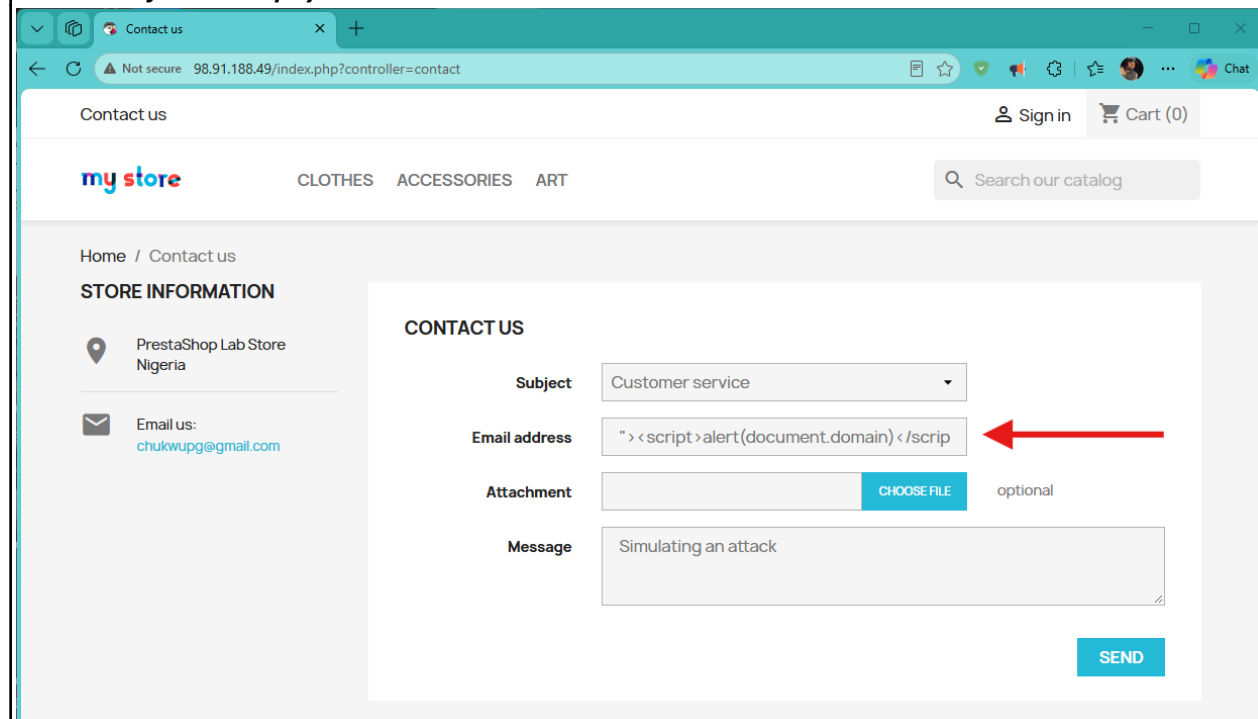
Findings: the same stored value was rendered inconsistently across the single thread detail page. The reply form heading and a hidden input field rendered the payload raw and unescaped. A separate strong tag context stripped angle brackets while preserving other characters. An h2 heading fully HTML entity encoded the same value. This confirms the underlying vulnerability class, improper output encoding, is present and inconsistently applied.

A follow up attempt to submit a fresh script tag payload specifically to test execution in the confirmed vulnerable heading location did not appear in the thread list or the underlying database. The cause was not conclusively isolated within the testing window, with CSRF token staleness and thread grouping by email as candidate explanations.

Impact: confirmed that an unauthenticated external party can inject content into the admin back office that is rendered without escaping in at least two page locations. Full script execution was not conclusively reproduced within the testing window, but the unescaped rendering itself confirms the vulnerability is present, consistent with the published CVE. Because HttpOnly is correctly set on session cookies on this instance, the specific session cookie theft impact described in the original advisory is partially mitigated here.

Mitigation: update PrestaShop to 8.2.6 or later. Ensure consistent output encoding is applied to all user controllable data rendered in back office views, not only the specific field patched upstream. The existing HttpOnly cookie configuration is a useful compensating control that should be retained.

Contact Us form with payload entered

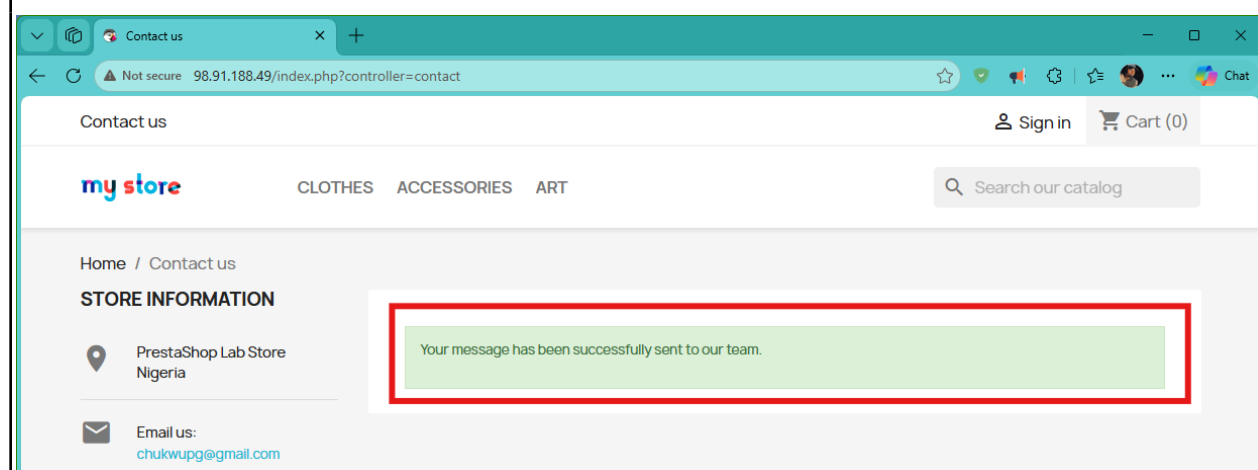


The screenshot shows a web browser window with the URL `98.91.188.49/index.php?controller=contact`. The page is titled "Contact us" and features a "my store" logo with navigation links for "CLOTHES", "ACCESSORIES", and "ART". A search bar is present with the text "Search our catalog". The left sidebar displays "STORE INFORMATION" for "PrestaShop Lab Store Nigeria" with an email address "chukwupg@gmail.com". The main content area is titled "CONTACT US" and contains a form with the following fields:

- Subject:** A dropdown menu with "Customer service" selected.
- Email address:** A text input field containing the payload `"><script>alert(document.domain)</scrip`. A red arrow points to this field.
- Attachment:** A text input field, a "CHOOSE FILE" button, and the word "optional".
- Message:** A text area containing the text "Simulating an attack".

A "SEND" button is located at the bottom right of the form.

Accepted submission confirmation



The screenshot shows the same "Contact us" page as above. The form fields are now empty. A green confirmation message box is displayed in the center of the page, containing the text: "Your message has been successfully sent to our team." The message box is highlighted with a red rectangular border.

5.5 Vector 2, No Brute Force Protection on Admin Login

Root cause: checklist item nine confirmed the admin login path has no IP restriction and no WAF in front of it. This vector tests whether the application itself compensates with rate limiting or lockout.

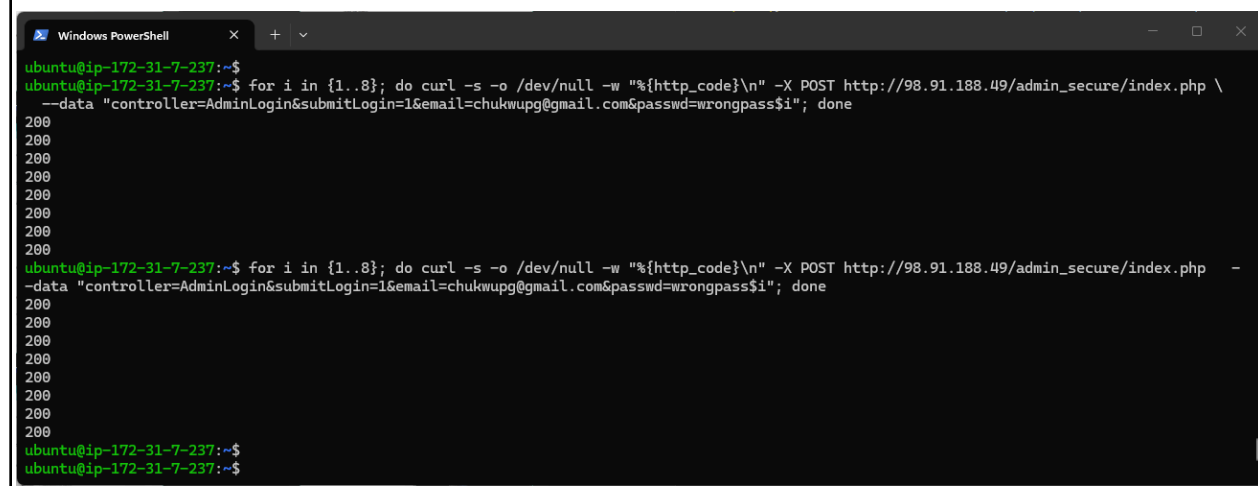
Methodology: repeated failed login attempts were submitted against a known valid admin account, both through the browser interface and through a scripted loop of automated requests, for a combined total of sixteen attempts.

Findings: all sixteen attempts returned identical 200 OK responses with a generic invalid password result. No CAPTCHA challenge, progressive delay, temporary account lockout, or alerting of any kind was triggered at any point.

Impact: combined with the absence of an IP restriction or WAF on the admin panel, this login endpoint has no application level friction whatsoever against sustained automated credential stuffing or brute force attacks. Current password strength is the sole barrier protecting this account, with no defense in depth, and the absence of multi factor authentication means a successful brute force would grant immediate, unrestricted back office access.

Mitigation: implement account lockout after a small number of failed attempts, with exponential backoff or a time based cooldown. Add a CAPTCHA after a small number of failed attempts. Restrict the admin path to a trusted IP allowlist at the infrastructure level. Deploy a WAF capable of rate limiting login attempts at the proxy layer. Add native multi factor authentication or a dedicated module once available.

Repeated failed login attempts with no lockout behavior simulated in terminal, output showing the scripted login attempts



```
Windows PowerShell
ubuntu@ip-172-31-7-237:~$
ubuntu@ip-172-31-7-237:~$ for i in {1..8}; do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://98.91.188.49/admin_secure/index.php \
--data "controller=AdminLogin&submitLogin=1&email=chukwupg@gmail.com&passwd=wrongpass$i"; done
200
200
200
200
200
200
200
200
ubuntu@ip-172-31-7-237:~$ for i in {1..8}; do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://98.91.188.49/admin_secure/index.php \
--data "controller=AdminLogin&submitLogin=1&email=chukwupg@gmail.com&passwd=wrongpass$i"; done
200
200
200
200
200
200
200
200
ubuntu@ip-172-31-7-237:~$
ubuntu@ip-172-31-7-237:~$
```

5.6 Vector 3, Plaintext Credential Interception, No TLS

Root cause: checklist item eight confirmed the instance serves all traffic over plain HTTP, with port 443 not listening and no TLS certificate present.

Methodology: packet capture was performed using Wireshark during a normal, legitimate admin login to the back office, simulating what any network positioned observer would be able to see without any application level exploitation.

Findings: the captured traffic contained the admin login request with the password field fully visible in plaintext, with no encryption applied at any point in transit.

Impact: this is the most severe and simplest of the three vectors to execute, since it requires no payload crafting, no authentication bypass, and no application level vulnerability at all, only network position. Any admin login performed over this instance's current configuration is fully compromised the moment an attacker can observe the traffic path.

Mitigation: configure TLS, using Let's Encrypt and Certbot if a domain is available, or a self signed certificate as an interim step for a lab environment, and enforce a redirect from HTTP to HTTPS. Enable the PS_SSL_ENABLED setting once TLS is live, and set the Secure flag on all session cookies, which also resolves checklist item five as a direct consequence.

Wireshark capture showing the plaintext password field

The image shows a Wireshark packet capture window. The top pane displays a list of network packets. Packet 136 is selected, showing it is an HTTP POST request to /admin_secure/index.php. The bottom pane shows the details of this packet, specifically the HTML Form URL Encoded section. A red box highlights the 'passwd' field, which contains the value 'kCv2-zwDQm'. Other fields visible include 'ajax' (1), 'token' (empty), 'controller' (AdminLogin), 'submitlogin' (1), 'email' (chuaung@gmail.com), and 'redirect' (http://98.91.188.49/admin_secure/index.php?cont...).

No.	Time	Source	Destination	Protocol	Length	Info
1.	14.417524	192.168.1.6	98.91.188.49	TCP	54	9732 → 80 [FIN, ACK] Seq=1 Ack=1 Win=255 Len=0
1.	14.417812	192.168.1.6	98.91.188.49	TCP	1506	16545 → 80 [ACK] Seq=1 Ack=1 Win=255 Len=1452 [TCP PDU reassembled in 136]
1.	14.417812	192.168.1.6	98.91.188.49	TCP	482	16545 → 80 [PSH, ACK] Seq=1453 Ack=1 Win=255 Len=428 [TCP PDU reassembled in 136]
1.	14.417868	192.168.1.6	98.91.188.49	HTTP	286	POST /admin_secure/index.php?rand=1789907449335 HTTP/1.1 (application/x-www-form-urlencoded)
1.	14.641875	98.91.188.49	192.168.1.6	TCP	54	80 → 9732 [ACK] Seq=1 Ack=2 Win=631 Len=0
1.	14.643805	98.91.188.49	192.168.1.6	TCP	54	80 → 16545 [ACK] Seq=1 Ack=1881 Win=610 Len=0

Frame 136: Packet, 286 bytes on wire (2288 bits), 286 bytes captured (2288 b...)

Ethernet II, Src: 7a:56:59:e3:f8:85 (7a:56:59:e3:f8:85), Dst: zte_e0:a4:d8 (...

Internet Protocol Version 4, Src: 192.168.1.6, Dst: 98.91.188.49

Transmission Control Protocol, Src Port: 16545, Dst Port: 80, Seq: 1881, Ack... [3 Reassembled TCP Segments (2112 bytes): #134(1452), #135(428), #136(232)]

Hypertext Transfer Protocol

HTML Form URL Encoded: application/x-www-form-urlencoded

- Form item: "ajax" = "1"
- Form item: "token" = ""
- Form item: "controller" = "AdminLogin"
- Form item: "submitlogin" = "1"
- Form item: "passwd" = "kCv2-zwDQm"
- Form item: "email" = "chuaung@gmail.com"
- Form item: "redirect" = "http://98.91.188.49/admin_secure/index.php?cont..."

5.7 Findings Considered But Not Used as Attack Vectors

The remaining checklist findings were reviewed and deliberately not pursued as standalone attack vectors, since they represent hygiene or resilience gaps rather than directly exploitable weaknesses. The presence of default demo catalog data is a reconnaissance signal rather than something to exploit directly. The missing Secure cookie flag is a consequence of the absent TLS finding rather than an independent vulnerability. The default module set includes no specific vulnerable module. Untested backup and recovery affects impact severity after a breach rather than providing a way to cause one.

5.8 Summary of Findings

Vector	Status	Severity
1. Stored Cross Site Scripting, CVE 2026 44212	Vulnerability class confirmed, full execution inconclusive	Critical
2. No brute force protection on admin login	Fully confirmed and reproducible	High
3. Plaintext credential interception, no TLS	Fully confirmed	Critical

5.9 Overall Conclusion

This lab instance benefits from solid infrastructure level hardening which was already configured in the existing Prestashop deployment I used; including SSH restriction, file permissions, and a disabled Webservice and API surface, but has three genuine and demonstrable weaknesses stemming directly from an outdated application version, absent authentication friction, and a complete lack of transport encryption. None of these required sophisticated tooling to exploit, only a public CVE with a documented technique, a handful of scripted login attempts, and a passive packet capture. Remediation priority is recommended as follows: first, enable TLS immediately, since it is the simplest fix with the broadest impact reduction, second, update PrestaShop to 8.2.6 or later, and third, add login rate limiting and lockout while restricting admin access by IP address.

5.10 Status of Mitigation and Re Test

Mitigation and re testing for all three vectors are planned as a follow up activity and will be documented separately once completed, following the same before and after validation approach used for the DVWA track in section 3.